

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

Error Analysis Task

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0405

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I *Yoga Madha A-192425058* (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

QUESTIONS:

Total Marks:25

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

a) Identify the conceptual error in the likelihood formulation.

Solution:

The student incorrectly defines the likelihood function as the **sum of probabilities** instead of the **product of probabilities**. In Maximum Likelihood Estimation (MLE), training samples are assumed to be **independent**, so the probability of observing the entire dataset is the **product of the individual probabilities**.

Incorrect:

$$L(\theta) = \sum_{i=1}^n P(x_i | \theta)$$

Correct:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

Thus, the conceptual error is violating the independence assumption of the observations.

b) Explain how this mistake affects parameter estimation.

Solution:

Using the sum instead of the product changes the optimization objective. The estimated parameters no longer maximize the probability of observing the complete dataset.

As a result:

- Incorrect parameter estimates are obtained.
- The classifier learns an inaccurate decision boundary.
- Prediction accuracy decreases.
- Even with large datasets, the model does not converge to the true parameter values because the optimization criterion is incorrect.

c) Suggest the correct mathematical formulation and reasoning.

Solution:

For a binary classifier, the likelihood function is

$$L(\theta) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{(1-y_i)}$$

where

- $p_i = P(y_i = 1 | x_i, \theta)$
- y_i is the class label (0 or 1).

This formulation correctly combines the probability of every independent observation.

Maximizing this likelihood provides the parameter values that make the observed data most probable.

d) How would using log-likelihood fix computational issues?

Solution:

The log-likelihood is

$$\log L(\theta) = \sum_{i=1}^n \log P(x_i | \theta)$$

Taking the logarithm converts the product into a sum.

Advantages:

- Prevents numerical underflow caused by multiplying many very small probabilities.
- Simplifies differentiation during optimization.
- Makes Gradient Descent and other optimization algorithms more efficient.
- Produces the same optimal parameter values because the logarithm is a monotonic function.

2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

a) List possible implementation errors in the learning algorithm.

Solution:

If a Perceptron fails to converge on a linearly separable dataset, possible implementation errors include:

- Incorrect weight update rule.
- Missing or incorrect bias term.
- Wrong activation or prediction function.
- Incorrect label encoding (should be +1 and -1).
- Errors in the learning rate implementation.
- Incorrect stopping condition or update logic.

b) Analyze how incorrect learning rate or weight updates affect convergence.

Solution:

The Perceptron updates its weights using:

$$w_{new} = w_{old} + \eta(y - \hat{y})x$$

where:

- w = weight vector
- η = learning rate
- y = actual output
- $\hat{y}$ = predicted output
- x = input vector

Analysis:

- A **high learning rate** causes large weight updates, making the decision boundary oscillate and preventing convergence.
- A **low learning rate** results in very slow learning and requires many iterations.
- Incorrect weight updates move the decision boundary in the wrong direction, causing continuous classification errors.

c) Identify dataset-related issues that may cause this problem.

Solution:

Possible dataset-related problems include:

- Incorrect or inconsistent class labels.
- Features with different scales (no normalization).
- Noisy or duplicate training samples.
- Data preprocessing errors.
- The dataset is not truly linearly separable due to labeling mistakes or noise.

d) Propose modifications to ensure convergence.

Solution:

To ensure convergence:

- Implement the correct Perceptron learning rule.
- Include and update the bias term properly.
- Select an appropriate learning rate (e.g., 0.01 or 0.1).
- Normalize or standardize input features.
- Verify correct class label encoding (+1 and -1).
- Shuffle training samples before each epoch.
- Remove noisy or mislabeled data and confirm that the dataset is linearly separable.

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

a) Identify possible mathematical or coding errors in gradient computation.

Solution:

A continuously increasing loss during training may result from errors in gradient computation, such as:

- Incorrect implementation of the backpropagation equations.
- Wrong derivative of the activation function.
- Incorrect sign in the weight update rule.
- Learning rate set too high.
- Errors in matrix multiplication or tensor dimensions.
- Incorrect initialization of weights or biases.

b) Explain the role of activation functions in gradient flow.

Solution:

Activation functions determine how gradients are propagated through the network during backpropagation.

- **Sigmoid** and **Tanh** can produce very small gradients, slowing learning in deep networks.
- **ReLU** helps maintain stronger gradients and speeds up convergence.
- Choosing an appropriate activation function improves gradient flow and training stability.

c) Analyze how vanishing or exploding gradients affect training.

Solution:

Vanishing Gradient

- Gradients become extremely small.
- Early layers receive almost no updates.
- Training becomes very slow or stops completely.

Exploding Gradient

- Gradients become extremely large.
- Weight updates become unstable.
- Loss increases rapidly and the model fails to converge.

Both problems reduce model accuracy and prevent effective learning.

d) Suggest techniques to fix the issue (e.g., normalization, initialization).

Solution:

To improve training stability:

- **Normalize input data** using standardization or batch normalization.
- **Use proper weight initialization** such as Xavier or He initialization.
- **Use ReLU or its variants** instead of sigmoid in deep networks.
- **Reduce the learning rate** if updates are unstable.
- **Apply gradient clipping** to prevent exploding gradients.
- **Use Batch Normalization** to stabilize gradient flow and speed up convergence.

4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

a) Explain why batch gradient descent is slow in large datasets.

Solution:

Batch Gradient Descent computes the gradient using **all training samples** before updating the model parameters. For large datasets, each update requires processing the entire dataset, making training slow and computationally expensive. Although the updates are stable, convergence takes more time.

b) Analyze the cause of fluctuations in SGD.

Solution:

Stochastic Gradient Descent (SGD) updates the model using **one training sample at a time**. Since each sample provides only an estimate of the true gradient, the updates are noisy, causing the loss to fluctuate during training. While SGD converges faster initially, its optimization path is less smooth than Batch Gradient Descent.

c) Compare the error behavior of Mini-Batch Gradient Descent.

Solution:

Mini-Batch Gradient Descent uses a **small batch of training samples** (e.g., 32 or 64) for each update.

- Faster than Batch Gradient Descent because fewer samples are processed per update.
- More stable than SGD because gradients are averaged over a batch.
- Produces smaller loss fluctuations while maintaining good convergence speed.
- Offers a good balance between computational efficiency and accuracy.

d) Recommend the best approach and justify with error trade-offs.

Solution:

Mini-Batch Gradient Descent is the preferred approach because it balances the advantages of Batch Gradient Descent and SGD.

Justification:

- Faster convergence than Batch Gradient Descent.
- More stable updates than SGD.
- Lower computational cost.
- Better generalization with reduced error fluctuations.
- Efficiently utilizes modern GPU hardware.

5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

a) Explain how high dimensionality increases model error.

Solution:

High-dimensional data contains a large number of features, making the model more complex. As the number of features increases, the available training data becomes sparse, causing the model to learn noise instead of meaningful patterns. This increases the risk of **overfitting** and leads to poor performance on unseen data.

b) Identify signs of overfitting in this scenario.

Solution:

The following signs indicate overfitting:

- Very high accuracy on the training dataset.
- Low accuracy or high error on the test dataset.

- Large gap between training and testing performance.
- The model memorizes the training data instead of learning general patterns.

c) Analyze why distance-based learning fails in high dimensions.

Solution:

Distance-based algorithms such as **K-Nearest Neighbors (KNN)** rely on measuring distances between data points. In high-dimensional spaces:

- Data points become sparse.
- Distances between points become very similar.
- It becomes difficult to distinguish between near and far neighbors.
- As a result, classification and prediction accuracy decrease.

d) Suggest dimensionality reduction or regularization techniques to improve performance.

Solution:

To reduce overfitting and improve model performance:

- **Principal Component Analysis (PCA):** Reduces the number of features while preserving important information.
- **Feature Selection:** Removes irrelevant and redundant features.
- **Regularization (L1/Lasso or L2/Ridge):** Reduces model complexity by limiting large weights.
- **Cross-Validation:** Helps select the best model and improves generalization.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand